

DagDB: A Zero-Copy 6-Bounded Ranked-DAG Substrate on Apple-Silicon Unified Memory

N. Matevosyan

2026-04-18

Abstract

We present DagDB, a compact ranked directed-acyclic-graph substrate for Apple-silicon unified memory, together with a four-stage compute pipeline and two engineering compiler passes. Each node is 64 bytes with fan-in bounded to 6 so that a LUT6 truth table fits exactly in one `uint64_t` register and the combinational step becomes `state = (lut >> idx) & 1` — a branch-free shift-and-mask. Storage is `MTLResourceOptions.storageModeShared` for zero-copy CPU GPU access; Morton Z-curve memory ordering is used for cache coalescing; rank monotonicity on edges enables a lock-free rank-by-rank dispatch. The full four-stage pipeline (forward wave, leaf mirror, backward wave, queen mirror) executes one tick in ≈ 5 ms on a six-node demonstration DAG on an Apple M5 Max; an early-stage 1-million-node reference runs at **0.71** GCUPS on the same device.

We describe two compiler passes that keep the hardware strictly within spec on realistic inputs: (i) a routing-tree pass that flattens forward fan-out > 6 into 6-ary trees of identity-LUT proxies, restoring fan-in symmetry under edge inversion; and (ii) an XOR-mask frustration-propagation scheme that gives the backward wave a real mutation source between epochs. A failure-boundary experiment on random k -SAT documents where the pipeline stalls as constraint density crosses the satisfiability threshold.

Two predecessor projects — a hex-grid Lotka–Volterra simulator (*Savanna*) and a greedy monotone state-alignment primitive (*iso-walk v2*) — informed DagDB’s design and are cited as separate work.

1 Intent

This is an HPC and systems paper describing hardware layout, Metal compute kernels, two compiler passes, and benchmark numbers for DagDB alone. Two separate predecessor projects (*Savanna*; *iso-walk v2* on the ribosome) are cited in §9 but not claimed as DagDB’s own numbers.

2 Mechanical sympathy: why these constraints

DagDB’s constraints are the reason its performance is what it is.

Fan-in ≤ 6 , LUT6 per node. A 6-input truth table has $2^6 = 64$ entries, each one bit, total 64 bits, which fits exactly in a single `uint64_t` register. A node’s combinational logic is then a single shift-and-mask against its own 64-bit table, with no conditional branching inside the kernel:

```
state = (lut >> index) & 1
```

No warp divergence, no instruction-cache misses on the hot path, no register spill. The fan-in bound is *what lets the kernel be branch-free*.

Morton Z-curve memory layout. At scale, the dominant cost on any graph computation is pointer-chase latency into DRAM. Morton (Z-order) linearisation puts spatial neighbours close in linear memory, so six-neighbour reads load shared cache lines. A $2.11\times$ speedup versus row-major was measured on a predecessor hex-grid substrate at 10^9 cells (§9); DagDB inherits the same layout as a design choice.

Unified memory (UMA). Apple-silicon systems expose DRAM physically shared between CPU and GPU, with `MTLResourceOptions.storageModeShared` giving zero-copy access in both directions. No `memcpy` barrier between CPU-side graph construction and GPU-side kernel dispatch.

Rank monotonicity. Edges satisfy $\text{rank}(src) < \text{rank}(dst)$. The scheduler dispatches rank-by-rank; within a rank, every node is independent; across ranks, data flow is strictly acyclic. Combined with a lock-free 7-coloring of the hex adjacency, this lets the whole substrate run parallel and lock-free on the GPU.

None of these is a new invention. The contribution is in how cleanly they compose into a 64-byte-per-node, branch-free, zero-copy, lock-free, cache-coalesced kernel on a laptop.

3 DagDB substrate

Each node occupies 64 bytes:

field	bytes	notes
<code>id</code>	4	unique
<code>rank</code>	4	leaves = 1, Queen = N
<code>inputs[6]</code>	24	signed indices, -1 = absent, fan-in ≤ 6
<code>lut</code>	8	64-bit LUT6 truth table, one register
<code>state</code>	1	ternary $\{-1, 0, +1\}$
<code>active</code>	1	wave-front flag
<code>padding</code>	22	cache-line alignment

The buffer is sized $2 \cdot \text{nodeCount}$: a forward half (indices $[0, \text{nodeCount})$) and a mirror half (indices $[\text{nodeCount}, 2 \cdot \text{nodeCount})$). The mirror of node i lives at index $i + \text{nodeCount}$ and at rank $2N - r + 1$ relative to the forward node’s rank r . Storage is `storageModeShared`; the same bytes are visible to CPU code (graph construction) and GPU kernels (execution) without copies.

4 Four-stage pipeline

One tick runs four Metal kernels plus instrumentation, dispatched one thread per node with a rank-band predicate:

- Forward wave** (ranks $1..N$, ascending): each node packs its 6-bit input index from $\mathbb{K}[\text{input}_k.\text{state} > 0]$, then emits $(\text{lut} \gg \text{idx}) \ \& \ 1$ as its new state. Branch-free.
- Leaf mirror** (at rank N): boundary copy into the mirror half.

3. **Backward wave** (mirror ranks $N + 1..2N + 1$): currently mirror-propagation; §6 upgrades this to real aggregation.
4. **Queen mirror**: single-thread dispatch; copies mirror queen state back to Queen.

An entropy sweep kernel accumulates $\sum_i \mathbb{I}[s_i \neq 0] \cdot \log_2(1 + \deg_i)$ in a fixed-point atomic counter for instrumentation only.

5 Compiler pass 1: routing trees (symmetry under inversion)

A real Boolean circuit has bounded fan-in (for us, 6) but *unbounded* fan-out: one leaf may feed many gates. The mirror half’s backward wave, if it is to aggregate validation information through a mirrored copy of the DAG with the same fan-in constraint, will face fan-in equal to the forward fan-out — typically much more than 6. The hardware forbids this.

Engineering response: a compiler pass flattens every forward fan-out $k > 6$ into a 6-ary tree of identity-LUT proxies inserted at intermediate ranks. A node u with fan-out 36 is rewritten as:

- u feeds six proxies $p_1, \dots, p_6$;
- each p_i feeds six original consumers ($6 \times 6 = 36$);
- proxy LUT is `0xAAAAAAAAAAAAAAAA` — functionally transparent (output = input bit 0).

Depth of the added tree is $\lceil \log_6 k \rceil$; overhead is $O(k/5)$ proxy nodes per fan-out-violating vertex. Ranks of downstream consumers are shifted by the proxy depth in a single $O(V + E)$ relabelling pass.

Invert edges for the mirror, and the tree mirrors to a 6-ary aggregation tree: each mirror consumer sees at most 6 upstream mirror-proxies, which in turn see at most 6 upstream mirror-nodes, etc. Symmetric by construction.

6 Compiler pass 2: frustration propagation (dynamics)

The current backward wave is a pure memory copy: `mirror[i].state = forward[i].state`. On fixed inputs, epoch $k + 1$ is bit-identical to epoch k : $dH/dt = 0$ for all $t > 1$. The pipeline is at present a feed-forward evaluator in a `while` loop.

Engineering response: turn the backward wave into a *frustration* computation. At each mirror node m paired with forward node f :

$$m.state = f.state \text{ XOR } target_bit$$

where `target_bit` is the mirror-node’s aggregated consumer-side expectation, carried forward from the queen mirror’s fold. Nonzero XOR = disagreement. This is the “frustration mask.”

When the frustration mask reaches a leaf mirror (rank $2N$), the host daemon reads it from UMA and either:

- **Engineering variant**: triggers a library-loader invocation — injects a new subgraph primitive into a designated patch region at the next forward wave.
- **Stochastic variant**: executes a Boltzmann-like bit-flip on the leaf proportional to local XOR density.

Either way, epoch $k + 1$ is *not* bit-identical to epoch k . Hardware-level discrete-logic analog of backpropagation: compare expected to observed, propagate disagreement toward inputs, mutate there.

7 DagDB benchmarks

All numbers here are DagDB’s own. Predecessor benchmarks are *not* claimed as DagDB performance and are referenced in §9.

Pipeline tick

Six-node demonstration DAG (three rank-0 leaves, two rank-1 gates, a rank-2 Queen) on Apple M5 Max:

metric	value
one full four-stage tick	≈ 5 ms
wave-lines processed per tick	6
entropy (fixed-point $\times 1000$)	3000

Queen post-forward-wave state persists through all four stages (correctness property). The engine does not yet mutate state across epochs on fixed inputs (this is the motivation for §6).

DagDB runtime (early-stage)

metric	value on Apple M5 Max
throughput on 10^6 -node graph	0.71 GCUPS
unit-test suite	27/27 passing
snapshot format (<code>.dags</code>)	zlib-compressed to $\approx 4\%$ of raw on sparse graphs
example compression (10^7 nodes)	358 MB $\rightarrow$ 14.4 MB

These are early-stage numbers on a small graph. Scaling to the 10^{10} -edge regime and layering in the routing-tree compiler (§5) and frustration propagation (§6) is future work.

8 Failure-boundary study

Any claim of convergence by a polynomial-budget solver on NP-hard input would imply $\mathbf{P} = \mathbf{NP}$. We therefore do not claim convergence. Instead:

- Target: random k -SAT at $k = 3$, sweeping the constraint-to-variable ratio α across the satisfiability threshold $\alpha_c \approx 4.267$.
- Observable: the clock-tick at which the pipeline exhausts its polynomial library-loader budget $p(n)$ without finding a witness.
- Claim: the exhaustion rate rises sharply as α crosses α_c , matching the CSP phase-transition literature (Cheeseman–Kanefsky–Taylor 1991; Monasson et al. 1999).

A solver that halts predictably with “unreachable” when it hits its polynomial budget is more useful than one that claims convergence it cannot deliver.

9 Related work (design lineage)

DagDB descends from two working predecessor projects that are reported separately and whose numbers are *not* claimed here.

Savanna — hex-grid Lotka–Volterra. A hex-grid ternary Lotka–Volterra simulator on Apple M5 Max reaching 15.8 GCUPS peak at 6.7×10^7 cells (ten runs, Morton Z-curve layout) and, in a separate tiled existence run, 1.07×10^{11} cells, with a measured $2.11\times$ speedup from Morton Z-curve memory layout at 10^9 cells versus row-major. The Morton choice, unified-memory storage mode, 6-bounded neighbourhood, and lock-free 7-coloring were validated in Savanna first and carried forward into DagDB’s design. Reported in a separate short paper.

Iso-walk v2 — state-alignment primitive. A greedy monotone state-alignment walk with precomputed gradient cache and incremental active-set tracking, benchmarked on the 4V9D $\rightarrow$ 4V9C conformational transition of *Thermus thermophilus* 70S ribosome (14,906 shared backbone atoms, 106,512 edges, $k=6$ KNN plus MST edges). On Apple M2: 75.2ms wall time, $7.3\mu\text{s}$ per flip, 10,284 flips, zero monotonicity violations, exact target convergence. Reported in a separate short paper; the primitive itself is planned for porting into DagDB as a per-rank matching step (future work).

A companion theoretical draft on distributed reconstruction (“DRT v2”) and a separately planned eigencone paper treat the mathematical content that motivated the ranked-DAG design. Both are out of scope for this paper.

Acknowledgments

AI assistance. Portions of this manuscript — LaTeX formalisation, figure rendering, literature integration, and copyediting — were prepared with the assistance of AI systems (Anthropic Claude Opus 4.7 with 1M-token context; Google Gemini DeepThink). The AI systems are disclosed as tools, not listed as authors. All engineering claims, empirical measurements, and errors are the responsibility of the named human author.

Humble disclaimer. This is an amateur engineering project. The author is not an HPC professional and makes no competitive performance claims against production systems. The benchmarks reported here are reproducible from the companion repositories and are intended as single-laptop order-of-magnitude markers.

References

- [1] P. Cheeseman, B. Kanefsky, W. M. Taylor. Where the really hard problems are. *IJCAI-91*, 331–337, 1991.
- [2] R. Monasson, R. Zecchina, S. Kirkpatrick, B. Selman, L. Troyansky. Determining computational complexity from characteristic phase transitions. *Nature* 400:133–137, 1999.
- [3] G. M. Morton. *A computer oriented geodetic data base and a new technique in file sequencing*. IBM Technical Report, Ottawa, Canada, 1966.